AWS CLI

The Complete Beginner-to-Practical Guide

Install, configure, and use the AWS Command Line Interface to manage S3, EC2, IAM, EBS, VPC, CloudWatch, Lambda, and RDS — with a hands-on final project.

A structured classroom / self-study reference

Table of Contents

Chapter 1 — What is AWS CLI?

Chapter 2 — Installing AWS CLI

Chapter 3 — Creating an IAM User for CLI Access

Chapter 4 — Configuring AWS CLI

Chapter 5 — Understanding AWS Regions

Chapter 6 — S3 Practical: Buckets and Files

Chapter 7 — EC2 Practical: Launching a Virtual Server

Chapter 8 — IAM Practical: Users and Groups

Chapter 9 — EBS Practical: Attaching Storage

Chapter 10 — VPC Practical: Networking Basics

Chapter 11 — CloudWatch: Monitoring AWS

Chapter 12 — Lambda: Running Serverless Functions

Chapter 13 — RDS: Managing Databases

Chapter 14 — Final Mini Project: Host a Website on EC2

Chapter 15 — Cleaning Up and Avoiding Surprise Charges

Appendix A — Quick Command Reference

Appendix B — Common Errors and Fixes

CHAPTER 1

What is AWS CLI?

The Problem: Using AWS Through the Browser

Without AWS CLI, every action requires clicking through the browser console: open the browser, log in to AWS, navigate to the EC2 service, then click "Create Instance" and fill out several forms. For a single server this is fine, but it does not scale — repeating dozens of clicks for every server, bucket, or database becomes slow and error-prone.

The Solution: AWS CLI

The AWS Command Line Interface (CLI) lets you manage AWS services by typing commands instead of clicking through menus. A task that took ten clicks in the console can often be done with a single command:

EXAMPLE

```
aws ec2 run-instances
```

This one line launches a new EC2 virtual server.

Definition

AWS CLI is an official command-line tool from Amazon Web Services that lets you control and automate AWS services — such as S3, EC2, IAM, and Lambda — directly from a terminal, using text commands instead of the web console.

Why Use AWS CLI? (Advantages)

- **Faster than the Console** — one command instead of many clicks.
- **Automation** — commands can be saved and re-run any time.
- **Scripting** — combine commands into shell/Python scripts to build entire environments.
- **DevOps friendly** — works naturally with CI/CD pipelines, Terraform, and Jenkins.
- **No browser needed** — useful for servers, cron jobs, and remote/headless machines.

> **NOTE:** The AWS CLI does not replace the console — it complements it. Many engineers use the console to explore and the CLI to automate repeatable work.

CHAPTER 2

Installing AWS CLI

AWS CLI version 2 is available for Windows, macOS, and Linux. Pick the instructions for your operating system below.

Windows

1. Download the installer from the official AWS CLI page:

- https://aws.amazon.com/cli/

2. Run the downloaded **.msi** file and follow the setup wizard like any normal Windows program (Next → Next → Install → Finish).

macOS

Download and run the official **.pkg** installer:

```
TERMINAL
curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
sudo installer -pkg AWSCLIV2.pkg -target /
```

Linux

Download and unzip the installer, then run the install script:

```
TERMINAL
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
```

Verify Installation

Open **CMD**, **PowerShell**, or a Terminal window and run:

```
COMMAND
aws --version
```

Expected output (version numbers will vary):

```
OUTPUT
```

```
aws-cli/2.x.x Python/3.x.x <platform>
```

Explain to Students

- **aws** — the base command that starts every AWS CLI instruction.
- **--version** — a flag that prints the installed version instead of running an action.

Uninstalling (if needed)

Windows: remove it from "Add or Remove Programs" like any other app. macOS/Linux: delete the installed files, typically with:

TERMINAL

```
sudo rm -rf /usr/local/aws-cli
sudo rm /usr/local/bin/aws
```

CHAPTER 3

Creating an IAM User for CLI Access

> **CAUTION:** Never use the AWS **Root User** for daily work or CLI access. The root user has unrestricted access to the entire account, including billing. Create a separate IAM user instead.

Step 1 — Open IAM

Go to the AWS Console:

- https://aws.amazon.com/console/

Search for **IAM** in the top search bar and open it.

Step 2 — Create the User

- Click **Users** in the left sidebar.
- Click **Create User**.
- Enter a username, for example: **cli-user**.
- Click **Next**.

Step 3 — Attach a Policy

For learning purposes, you can attach **AdministratorAccess** directly. Then click **Create User**.

> **NOTE:** AdministratorAccess grants full permissions to every AWS service. This is fine for a personal practice account, but in a real team environment you should grant only the specific permissions a user needs (the "principle of least privilege").

Step 4 — Create an Access Key

- Open the newly created user.
- Go to the **Security Credentials** tab.
- Click **Create Access Key**.
- Select the use case **Command Line Interface (CLI)**.
- Click **Generate Key**.

You will receive two values:

- **Access Key ID**
- **Secret Access Key**

CAUTION: The Secret Access Key is shown only once. Download the .csv file or copy both values somewhere safe immediately — you cannot view the secret key again later.

CHAPTER 4

Configuring AWS CLI

Open a terminal (PowerShell, CMD, or Terminal) and run:

COMMAND

```
aws configure
```

You will be prompted for four values, one at a time:

EXAMPLE INPUT

```
AWS Access Key ID [None]: AKIA123456
AWS Secret Access Key [None]: abc123xyz
Default region name [None]: ap-south-1
Default output format [None]: json
```

What Each Value Means

- **AWS Access Key ID** — identifies which IAM user is connecting.
- **AWS Secret Access Key** — the password paired with that Access Key ID. Keep it private.
- **Default region** — which AWS data center your commands will target by default (e.g. **ap-south-1** for Mumbai).
- **Default output format** — how command results are displayed: **json**, **text**, or **table**.

Verify Login

Confirm the CLI is authenticated correctly:

COMMAND

```
aws sts get-caller-identity
```

OUTPUT

```
{
"UserId": "AIDAEXAMPLE123",
"Account": "123456789012",
"Arn": "arn:aws:iam::123456789012:user/cli-user"
}
```

Explain

STS stands for **Security Token Service**. This command asks AWS "who am I logged in as right now?" and is the standard way to confirm the CLI is correctly configured.

> **NOTE:** Credentials are stored locally in a hidden folder: **~/.aws/credentials** on macOS/Linux, or **%USERPROFILE%\.aws\credentials** on Windows. Never commit this folder or these keys to GitHub or any public repository.

CHAPTER 5

Understanding AWS Regions

List every AWS region available to your account:

COMMAND

```
aws ec2 describe-regions
```

Simplified output — a list of region codes:

OUTPUT

```
ap-south-1
us-east-1
us-west-2
eu-west-1
```

What Is a Region?

A **Region** is a physical AWS data center location. Choosing a region close to your users reduces latency, and some services or prices vary by region.

Common Region Codes

Region Code	Location
ap-south-1	Mumbai, India
ap-southeast-1	Singapore
us-east-1	N. Virginia, USA
us-west-2	Oregon, USA
eu-west-1	Ireland

CHAPTER 6

S3 Practical: Buckets and Files

S3 (Simple Storage Service) is AWS's file storage service. Files are stored inside containers called **buckets**.

List Buckets

COMMAND

```
aws s3 ls
```

Shows every bucket in your account.

Create a Bucket

COMMAND

```
aws s3 mb s3://hari-demo-bucket
```

mb stands for **Make Bucket**.

> **NOTE:** S3 bucket names must be globally unique across all AWS accounts worldwide. If the name is taken, choose a more specific one (e.g. add your name or a random number).

Verify the Bucket

COMMAND

```
aws s3 ls
```

Create a Sample File

COMMAND

```
echo Hello AWS > file.txt
dir (Windows)
ls (macOS/Linux)
```

Upload a File

COMMAND

```
aws s3 cp file.txt s3://hari-demo-bucket
```

cp stands for **Copy** — it copies the local file to S3.

View Files in the Bucket

COMMAND

```
aws s3 ls s3://hari-demo-bucket
```

Download a File

COMMAND

```
aws s3 cp s3://hari-demo-bucket/file.txt .
```

The dot (**.**) means "download into the current folder".

Sync an Entire Folder

COMMAND

```
mkdir backup
aws s3 sync backup s3://hari-demo-bucket
```

sync uploads only new or changed files, making it efficient for repeated backups — unlike **cp**, it will not re-upload files that already match.

CHAPTER 7

EC2 Practical: Launching a Virtual Server

EC2 (Elastic Compute Cloud) lets you rent virtual servers in the cloud. This chapter walks through creating everything a server needs: an image, a login key, a firewall rule, and finally the server itself.

List Available AMIs

COMMAND

```
aws ec2 describe-images --owners amazon
```

An **AMI (Amazon Machine Image)** is a template used to create a server — similar to a Windows installation ISO or a cloned hard disk.

Create a Key Pair

A key pair is used to securely log in to your server (like an SSH password).

COMMAND

```
aws ec2 create-key-pair --key-name mykey \
--query KeyMaterial --output text > mykey.pem
```

CAUTION: On macOS/Linux, restrict permissions on the key file before using it, or SSH will refuse to use it: **chmod 400 mykey.pem**

Create a Security Group

A security group is a virtual firewall attached to your server.

COMMAND

```
aws ec2 create-security-group \
--group-name webserver \
--description "Web Server"
```

Output returns a security group ID, e.g. **sg-xxxxxxxx** — save it, you'll need it below.

Open the SSH Port (22)

COMMAND

```
aws ec2 authorize-security-group-ingress \
--group-id sg-xxxx \
--protocol tcp --port 22 --cidr 0.0.0.0/0
```

Open the HTTP Port (80)

COMMAND

```
aws ec2 authorize-security-group-ingress \
--group-id sg-xxxx \
--protocol tcp --port 80 --cidr 0.0.0.0/0
```

CAUTION: 0.0.0.0/0 allows access from any IP address on the internet. This is fine for a short classroom demo, but for real systems restrict SSH (port 22) to your own IP address only.

Launch the EC2 Instance

COMMAND

```
aws ec2 run-instances \
--image-id ami-xxxxxxxx \
--instance-type t2.micro \
--key-name mykey \
--security-group-ids sg-xxxx
```

NOTE: t2.micro is eligible for the AWS Free Tier on new accounts (750 hours/month for the first 12 months). Always confirm current Free Tier terms on the AWS pricing page before relying on it.

View Instance Details

COMMAND

```
aws ec2 describe-instances
```

Stop / Start / Terminate

STOP (KEEPS STORAGE, NO COMPUTE CHARGES)

```
aws ec2 stop-instances --instance-ids i-xxxx
```

START

```
aws ec2 start-instances --instance-ids i-xxxx
```

TERMINATE (PERMANENTLY DELETES THE INSTANCE)

```
aws ec2 terminate-instances --instance-ids i-xxxx
```

NOTE: Stop pauses billing for compute time but keeps the attached disk (small storage charges continue). **Terminate** permanently deletes the instance and cannot be undone.

CHAPTER 8

IAM Practical: Users and Groups

List Users

COMMAND

```
aws iam list-users
```

Create a User

COMMAND

```
aws iam create-user --user-name student1
```

Create a Group

COMMAND

```
aws iam create-group --group-name developers
```

Add a User to a Group

COMMAND

```
aws iam add-user-to-group \
--user-name student1 \
--group-name developers
```

Groups let you manage permissions for many users at once: attach a policy to the group, and every member automatically inherits it.

CHAPTER 9

EBS Practical: Attaching Storage

Create a Volume

COMMAND

```
aws ec2 create-volume \
--size 10 \
--availability-zone ap-south-1a
```

Attach the Volume to an Instance

COMMAND

```
aws ec2 attach-volume \
--volume-id vol-xxxx \
--instance-id i-xxxx \
--device /dev/sdf
```

Explain

- **EBS (Elastic Block Store)** — a virtual hard disk.
- **EC2** — the computer that the disk is plugged into.
- Attaching EBS to EC2 is conceptually like plugging in a USB drive.

> **NOTE:** An EBS volume must be in the **same Availability Zone** as the EC2 instance it attaches to.

CHAPTER 10

VPC Practical: Networking Basics

List VPCs

COMMAND

```
aws ec2 describe-vpcs
```

Create a VPC

COMMAND

```
aws ec2 create-vpc --cidr-block 10.0.0.0/16
```

Understanding CIDR

A **VPC (Virtual Private Cloud)** is your own private, isolated network inside AWS. The **CIDR block** defines its IP address range.

10.0.0.0/16 allows approximately 65,536 IP addresses within that network.

> **NOTE:** A smaller number after the slash means a larger network. /16 is large (65,536 IPs), while /24 is smaller (256 IPs) — useful for subnets.

CHAPTER 11

CloudWatch: Monitoring AWS

View Available Metrics

COMMAND

```
aws cloudwatch list-metrics
```

Get CPU Utilization Statistics

COMMAND

```
aws cloudwatch get-metric-statistics \
--namespace AWS/EC2 \
--metric-name CPUUtilization \
--dimensions Name=InstanceId,Value=i-xxxx \
--start-time 2026-08-09T00:00:00Z \
--end-time 2026-08-10T00:00:00Z \
--period 3600 \
--statistics Average
```

What CloudWatch Monitors

- **CPU** usage — available by default.
- **RAM (memory)** usage — requires installing the CloudWatch Agent.
- **Disk** usage and I/O.
- **Network** traffic in and out.

CloudWatch is AWS's built-in monitoring service — it collects metrics, logs, and can trigger alarms (for example, emailing you if CPU usage stays above 90% for 5 minutes).

CHAPTER 12

Lambda: Running Serverless Functions

AWS Lambda runs your code without you having to manage any server. You upload a function, and AWS runs it only when triggered — you pay only for the time it actually runs.

List Functions

COMMAND

```
aws lambda list-functions
```

Invoke a Function

COMMAND

```
aws lambda invoke \
--function-name HelloWorld \
output.txt
```

Check the Result

COMMAND

```
type output.txt (Windows)
cat output.txt (macOS/Linux)
```

The invoke command calls **HelloWorld** and saves the returned response into **output.txt**.

CHAPTER 13

RDS: Managing Databases

RDS (Relational Database Service) is a managed database service — AWS handles backups, patching, and maintenance for you.

List Database Instances

COMMAND

```
aws rds describe-db-instances
```

Stop a Database

COMMAND

```
aws rds stop-db-instance \
--db-instance-identifier mydb
```

Start a Database

COMMAND

```
aws rds start-db-instance \
--db-instance-identifier mydb
```

NOTE: A stopped RDS instance still incurs storage charges, and AWS automatically restarts it after 7 days if left stopped. Terminate it instead if you no longer need it.

CHAPTER 14

Final Mini Project: Host a Website on EC2

This project combines everything covered so far: S3, EC2, and basic Linux server administration, to publish a live website.

Goal

- Create an S3 bucket
- Upload website files
- Launch an EC2 instance
- Install the Apache web server
- View the live website in a browser

1. Create the Bucket

COMMAND

```
aws s3 mb s3://student-demo
```

2. Upload the Website Files

COMMAND

```
aws s3 sync website s3://student-demo
```

3. Launch the EC2 Server

COMMAND

```
aws ec2 run-instances \
--image-id ami-xxxxxxxx \
--instance-type t2.micro \
--key-name mykey \
--security-group-ids sg-xxxx
```

4. Connect to the Server

COMMAND

```
ssh -i mykey.pem ec2-user@PUBLIC-IP
```

Replace **PUBLIC-IP** with the address shown by **describe-instances**.

5. Install and Start Apache

COMMAND (RUN ON THE EC2 SERVER)

```
sudo yum install httpd -y
sudo systemctl start httpd
sudo systemctl enable httpd
```

enable makes Apache start automatically every time the server reboots.

6. Create the Webpage

COMMAND

```
echo "AWS CLI Demo" > /var/www/html/index.html
```

7. View the Website

OPEN IN BROWSER

```
http://PUBLIC-IP
```

NOTE: If the page does not load, double-check that the security group allows inbound traffic on port 80 (from Chapter 7) and that Apache is running (**sudo systemctl status httpd**).

CHAPTER 15

Cleaning Up and Avoiding Surprise Charges

AWS bills for resources that are running or stored, even if you are not actively using them. After finishing practice exercises, clean up to avoid unexpected charges.

Checklist

- Terminate EC2 instances you no longer need: **aws ec2 terminate-instances --instance-ids i-xxxx**
- Delete unused EBS volumes: **aws ec2 delete-volume --volume-id vol-xxxx**
- Empty and delete S3 buckets: **aws s3 rb s3://bucket-name --force**
- Delete unused security groups and VPCs once instances are terminated.
- Stop or delete RDS databases you are not using.
- Check the AWS **Billing Dashboard** periodically for unexpected costs.

> **NOTE:** Set up a **Billing Alarm** in CloudWatch early on — it can email you the moment your estimated monthly charges cross a threshold you choose (e.g. $5).

CHAPTER A

Quick Command Reference

Task	Command
Check CLI version	`aws --version`
Configure credentials	`aws configure`
Verify identity	`aws sts get-caller-identity`
List regions	`aws ec2 describe-regions`
List S3 buckets	`aws s3 ls`
Create S3 bucket	`aws s3 mb s3://name`
Upload file to S3	`aws s3 cp file s3://bucket`
Sync folder to S3	`aws s3 sync folder s3://bucket`
Launch EC2 instance	`aws ec2 run-instances ...`
List EC2 instances	`aws ec2 describe-instances`
Stop / Start EC2	`aws ec2 stop-instances / start-instances`
Terminate EC2	`aws ec2 terminate-instances`
List IAM users	`aws iam list-users`
Create EBS volume	`aws ec2 create-volume`
List VPCs	`aws ec2 describe-vpcs`
List Lambda functions	`aws lambda list-functions`
Invoke Lambda function	`aws lambda invoke`
List RDS databases	`aws rds describe-db-instances`

CHAPTER B

Common Errors and Fixes

Error	Likely Cause / Fix
Unable to locate credentials	Run 'aws configure' and re-enter your Access Key ID and Secret Access Key.
An error occurred (UnauthorizedOperation)	The IAM user lacks permission for that action — attach the needed policy.
Bucket name already exists	S3 bucket names are global. Choose a more unique name.
InvalidKeyPair.NotFound	The --key-name does not match a key pair created in that region.
Connection timed out (SSH)	Security group is not allowing port 22, or the wrong IP/PUBLIC-IP was used.
command not found: aws	Installation did not complete, or PATH was not updated — reopen the terminal or reinstall.